



Higher Diploma in Computer Science

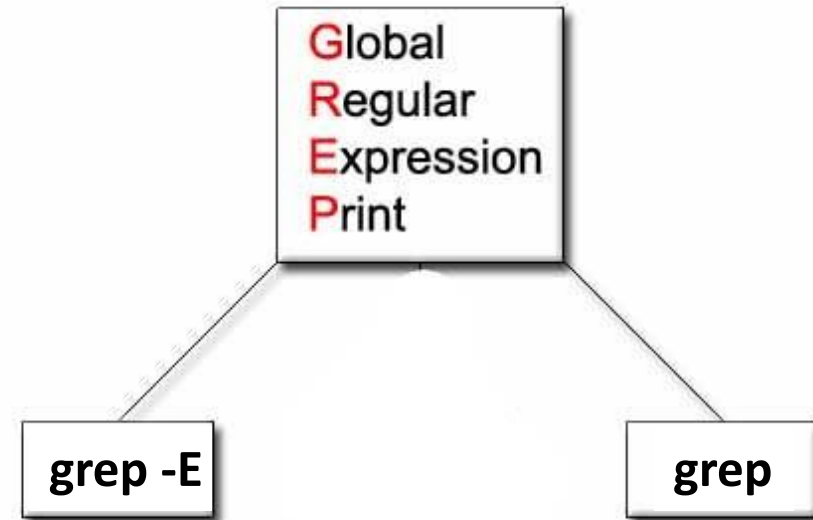
Computer Systems & Networks





Regular Expressions

```
grep 'example' fileName  
grep 'example' file1 file2 file3  
grep 'string1 string2' fileName  
cat otherfile | grep 'example2'  
command | grep 'example2'  
command option1 | grep 'data'  
grep --color 'data' filename
```



Regular Expressions(regex)

- *Regex* are a collection of *normal* and *special* characters used to find simple or complex patterns, respectively, in files.
 - 1) *Normal characters* are alphanumeric characters which match themselves. E.g. an *a* character would match an *a*
 - 2) *Metacharacters/Special characters* have special meanings when used within patterns by commands

Basic RegEx - The . Character

- The . character matches any character except for the new line character.
- The pattern `r..f` would find any line that contained the letter `r` followed by exactly two characters and then the letter `f`:

```
$ grep 'r..f' red.txt  
reef  
roof
```

- The . character can be used any number of times.

Basic RegEx - The [] Character

The square brackets [] match a single character from the list or range of possible characters contained within the brackets:

- “[0-9]” will match all lines that contain a digit
- “[aeiou]” will match all lines that contain a vowel.
- “[^xyz]” A caret can be used at the beginning of the range to specify a negative range. E.g. “[^xyz]” will match all lines that do not contain x, y or z

Each possible character *can* be listed out [abcd] or

provided as a range [a-d], as long as the range is in the correct order.

Basic RegEx - The * Character

- * is used to match zero or more occurrences of a character or pattern preceding it.
- For example, `e*` would match zero or more occurrences of the letter `e`:

```
sysadmin@localhost:~/Documents$ grep 're*d' red.txt
red
reed
rd
```

- It is also possible to match zero or more occurrences of a **list** of characters using the square brackets.

```
sysadmin@localhost:~/Documents$ grep 'r[oe]*d' red.txt
```

Basic RegEx - Anchor Characters

^ \$

Anchor characters specify whether the match occurs at the beginning of the line or the end of the line.

1. BEGINNING: The caret ^ character is used to ensure that a pattern appears at the beginning of the line:

```
sysadmin@localhost:~/Documents$ grep '^root' passwd  
root:x:0:0:root:/root:/bin/bash
```

2. END: The second anchor character, \$, can be used to ensure a pattern appears at the end of the line:

```
sysadmin@localhost:~/Documents$ grep 'r$' alpha-first.txt  
B is for Bear  
F is for Flower
```


SUMMARY

Character	Matches
.	Any single character
[]	A list or range of characters to match one character. If the first character is the caret ^, it means any character <i>not</i> in the list
*	The previous character repeated zero or more times
^	If the first character in the pattern, the pattern must be at the beginning of the line to match, otherwise just a literal ^
\$	If the last character in the pattern, the pattern must be at the end of the line to match, otherwise just a literal \$

Basic RegEx - The \ Escape Character

- Use the backslash character \ to match a character that happens to be a special Regular Expression character:

In the example below, the pattern `re*` matches an `r` (followed by zero or more of the letter `e`):

```
sysadmin@localhost:~/Documents$ grep 're*' newhome.txt
**Bewaree** of the ghost in the bedroom.
```

- To search for the asterisk `*` character, place a \ (backslash or escape character) before the `*` character:

```
sysadmin@localhost:~/Documents$ grep 're\*' newhome.txt
**Beware*** of the ghost in the bedroom.
```

Basic RegEx - The \ Escape Character

- In grep, these are treated as pattern instead of meta characters:

? + { } | ()

- By putting 'backslash' in front of meta character can let the grep to treat it as meta characters like

\? \+ \{ \| \(\ \)

Using **grep -E**, you wouldn't need to escape them



Extended Regular Expressions

Curly Brackets { }

- Tells us about the number of occurrences of a character in a string

Expression	Description	EG
{n}	Matches the preceding character appearing 'n' times exactly	\$cat sample grep -E p\{2}
{n,m}	Matches the preceding character appearing 'n' times but not more than m	
{n, }	Matches the preceding character only when it appears ' n' times or more	
\+	Matches one or more occurrence of the previous character	\$ cat employee.txt grep "w\+a"
\?	Matches zero or one occurrence of the previous character	

Extended RegEx

grep -E = egrep

- Use the `?` character to match `colo` followed by zero or one `u` character followed by an `r` character:

```
sysadmin@localhost:~/Documents$ grep -E 'colou?r' spelling.txt
American English: Do you consider gray to be a color or a shade?
British English: Do you consider grey to be a colour or a shade?
```

- Use the `+` character to match one or more `e` characters:

```
sysadmin@localhost:~/Documents$ grep -E 'e+' red.txt
red
reef
reed
```

- Use the `|` character to match either `gray` or `grey`:

```
sysadmin@localhost:~/Documents$ grep -E 'gray|grey' spelling.txt
American English: Do you consider gray to be a color or a shade?
British English: Do you consider grey to be a colour or a shade?
```

Summary

^	Indicates the beginning of an input string
\$	Indicates the end of the an input string
*	Indicates the preceding subexpression is to be matched zero or more times
+	Indicates the preceding subexpression is to be matched one or more times
?	Indicates the preceding subexpression is to be matched zero or one time
{n}	Match exactly n times (Where n is a non-negative integer)
{n,}	Match at least n times (Where n is a non-negative integer)
{n,m}	Match at least n and at most m times (Where m and n are non-negative integers and $n \leq m$)
.	Matches any single character except “n”
[xyz]	Match any one of the enclosed characters
x y	Match either x or y
[^xyz]	Match any character not enclosed
[a-z]	Matches any character in the specified range.
[^a-z]	Matches any character not in the specified range

Summary

b	Matches a word boundary (the position between a word and a space)
B	Matches a nonword boundary (example: 'uxB' matches the 'ux' in "tuxedo" but not the 'ux' in "Linux")
d	Matches a digit character
D	Matches a non-digit character
f	Matches a form-feed character
n	Matches a newline character
r	Matches a carriage return character
s	Matches any whitespace character (including space, tab, form-feed, etc.)
S	Matches any non-whitespace character
t	Matches a tab character
v	Matches a vertical tab character
w	Matches any word character including underscore
W	Matches any non-word character
un	Matches n, where n is a Unicode character expressed as four hexadecimal digits (such as, u0026 for the ampersand symbol)